

CHAP 1 : Généralités du langage SQL	2
Leçon 1 : Présentation du langage SQL.....	2
1. Historique du SQL.....	2
2. Différents types de SQL.....	2
Leçon 2 : La base de données relationnelle MySQL	3
1. Rôle	3
2. Historique	3
3. Installation de MariaDB avec XAMPP	4
CHAP 2 : LE LANGAGE DE DEFINITION DE DONNES (LDD).....	5
Leçon 1 : Création des tables.....	5
1. Commande de création d'une table	5
2. Type des données	6
3. Contraintes	7
4. Structure d'une table (DESCRIBE)	9
5. Commentaires	9
6. Exercice d'application	9
Leçon 2 : Modification de la structure des tables	11
1. Ajouter une colonne	11
2. Renommer une colonne.....	11
3. Modifier une colonne (type, taille, options)	12
4. Supprimer une colonne.....	12
5. Ajouter des contraintes.....	13
6. Exercice d'application	13
Leçon 3 : Les suppression dans le LDD	14
1. Supprimer une table	14
2. Supprimer les données d'une table	14
3. Exercice d'application	14

CHAP 1 : Généralités du langage SQL

Leçon 1 : Présentation du langage SQL

1. Historique du SQL

Le **SQL (Structured Query Language)** est un langage créé pour interagir avec les bases de données relationnelles.

- **1970** : Edgar F. Codd publie le modèle relationnel.
- **1974** : IBM développe SEQUEL (Structured English Query Language).
- **1979** : Oracle commercialise la première version de SQL.
- **1986** : Normalisation par l'ANSI (American National Standards Institute).
- **1987** : Normalisation par l'ISO.

Depuis, SQL évolue régulièrement (SQL-89, SQL-92, SQL:1999, SQL:2003, etc.).

Le SQL est aujourd'hui le **langage standard pour la gestion des données relationnelles** (MySQL, PostgreSQL, SQL Server, Oracle, MariaDB, etc.).

2. Différents types de SQL

SQL se divise en trois grandes catégories :

a) LDD — Langage de Définition de Données (DDL)

Permet de définir ou modifier la structure des tables.

Commandes : **CREATE** | **ALTER** | **DROP** | **TRUNCATE**

b) LMD — Langage de Manipulation de Données (DML)

Gère l'ajout, la modification et la suppression des données.

Commandes : **INSERT** | **UPDATE** | **DELETE** | **SELECT**

(souvent classé à part, mais lié au DML)

c) LCD — Langage de Contrôle de Données (DCL)

Gère les permissions et la sécurité.

Commandes : **GRANT** | **REVOKE** | **DENY** (SQL Server)

Leçon 2 : La base de données relationnelle MySQL

1. Rôle

MySQL est un **Système de Gestion de Base de Données Relationnelle (SGBDR)** open-source.

Il permet de créer, gérer, sécuriser et interroger des bases de données via le langage SQL.

Points forts :

- Très utilisé dans le Web (WordPress, PHP, Laravel...)
- Rapidité et robustesse
- Version gratuite disponible
- Compatible Linux, Windows, Mac
- Utilisé dans le modèle LAMP (Linux – Apache – MySQL – PHP)

2. Historique

- MySQL a été créé par deux ingénieurs suédois : **Michael “Monty” Widenius** et **David Axmark**.
- L'objectif initial était de développer un SGBD rapide pour remplacer un ancien système propriétaire qu'ils utilisaient.
- En **1995**, la première version publique de MySQL (Version **1.0**) est publiée sous licence **GPL**, ce qui attire très vite la communauté open source.
- En **2001**, sortie de **MySQL 3.23**, première version stable largement adoptée.
- En **2003**, la version **4.0** introduit les transactions et le moteur **InnoDB**, permettant une meilleure fiabilité.
- En **février 2008**, **Sun Microsystems** rachète MySQL AB, la société derrière MySQL, pour **1 milliard de dollars**.
- L'objectif de Sun : renforcer son offre dans les solutions open source.
- En **2010**, Oracle Corporation rachète Sun Microsystems.
- MySQL devient donc une propriété d'Oracle.

- Ce rachat suscite des inquiétudes car Oracle possède déjà un SGBD concurrent (Oracle Database).
- Pour rassurer la communauté, Oracle maintient MySQL en open source et continue son développement.
- Suite au rachat par Oracle, Monty Widenius quitte le projet et fonde **MariaDB**, un fork (dérivé) de MySQL.
- MariaDB devient l'alternative 100% libre à MySQL.
- De nombreuses distributions Linux migrent vers MariaDB (Debian, Fedora...).
- MySQL poursuit son développement sous Oracle, avec plusieurs améliorations

3. Installation de MariaDB avec XAMPP

CHAP 2 : LE LANGAGE DE DEFINITION DE DONNES (LDD)

Leçon 1 : Création des tables

Le LDD permet de créer, modifier et supprimer des objets dans une base de données.

1. Commande de création d'une table

La commande « **Create** » permet de créer une table, vue, index, etc.

Syntaxe :

```
CREATE TABLE nomTable(  
    colonne1    type1        [NOT NULL | NULL] [DEFAULT valeur1] ,  
    colonne2    type2        [NOT NULL | NULL] [DEFAULT valeur1] ,  
  
    colonneN    typeN        [NOT NULL | NULL] [DEFAULT valeur1]  
  
    );
```

Exemple:

```
CREATE TABLE Client (  
    id INT PRIMARY KEY,  
    nom VARCHAR(50),  
    ville VARCHAR(50)  
);
```

2. Type des données

Chaines de caractère

Type	Description	Commentaire pour une colonne
CHAR(<i>n</i>) [BINARY ASCII UNICODE]	Chaîne fixe de <i>n</i> octets ou caractères.	Taille fixe (maximum de 255 caractères).
VARCHAR(<i>n</i>) [BINARY]	Chaîne variable de <i>n</i> caractères ou octets.	Taille variable (maximum de 65 535 caractères).
BINARY(<i>n</i>)	Chaîne fixe de <i>n</i> octets.	Taille fixe (maximum de 255 octets).
VARBINARY(<i>n</i>)	Chaîne variable de <i>n</i> octets.	Taille variable (maximum de 255 octets).
TINYTEXT(<i>n</i>)	Flot de <i>n</i> octets.	Taille fixe (maximum de 255 octets).
TEXT(<i>n</i>)	Flot de <i>n</i> octets.	Taille fixe (maximum de 65 535 octets).
MEDIUMTEXT(<i>n</i>)	Flot de <i>n</i> octets.	Taille fixe (maximum de 16 mégaoctets).
LONGTEXT(<i>n</i>)	Flot de <i>n</i> octets.	Taille fixe (maximum de 4,29 gigaoctets).

Données numériques

Type	Description
BIT[(<i>n</i>)]	Ensemble de <i>n</i> bits. Taille de 1 à 64 (par défaut 1).
TINYINT[(<i>n</i>)] [UNSIGNED] [ZEROFILL]	Entier (sur un octet) de -128 à 127 signé, 0 à 255 non signé.
BOOL et BOOLEAN	Synonymes de TINYINT(1), la valeur zéro est considérée comme fausse. Le non-zéro est considéré comme vrai. Dans les prochaines versions, le type <i>boolean</i> , comme le préconise la norme SQL, sera réellement pris en charge.
SMALLINT[(<i>n</i>)] [UNSIGNED] [ZEROFILL]	Entier (sur 2 octets) de -32 768 à 32 767 signé, 0 à 65 535 non signé.
MEDIUMINT[(<i>n</i>)] [UNSIGNED] [ZEROFILL]	Entier (sur 3 octets) de -8 388 608 à 8 388 607 signé, 0 à 16 777 215 non signé.
INTEGER [(<i>n</i>)] [UNSIGNED] [ZEROFILL]	Entier (sur 4 octets) de -2 147 483 648 à 2 147 483 647 signé, 0 à 4 294 967 295 non signé.
BIGINT[(<i>n</i>)] [UNSIGNED] [ZEROFILL]	Entier (sur 8 octets) de -9 223 372 036 854 775 808 à 9 223 372 036 854 775 807 signé, 0 à 18 446 744 073 709 551 615 non signé.
FLOAT[(<i>n</i> , <i>p</i>)] [UNSIGNED] [ZEROFILL]	Flottant (de 4 à 8 octets) <i>p</i> désigne la précision simple (jusqu'à 7 décimales) de -3.4 10 ⁺³⁸ à -1.1 10 ⁻³⁸ , 0, signé, et de 1.1 10 ⁻³⁸ à 3.4 10 ⁺³⁸ non signé.
DOUBLE[(<i>n</i> , <i>p</i>)] [UNSIGNED] [ZEROFILL]	Flottant (sur 8 octets) <i>p</i> désigne la précision double (jusqu'à 15 décimales) de -1.7 10 ⁺³⁰⁸ à -2.2 10 ⁻³⁰⁸ , 0, signé, et de 2.2 10 ⁻³⁰⁸ à 1.7 10 ⁺³⁰⁸ non signé.

DECIMAL [(<i>n</i> [, <i>p</i>])] [UNSIGNED] [ZEROFILL]	Décimal à virgule fixe, <i>p</i> désigne la précision (nombre de chiffres après la virgule, maximum 30). Par défaut <i>n</i> vaut 10, <i>p</i> vaut 0.
---	--

Dates et Heures

Type	Description	Commentaire pour une colonne
DATE	Dates du 1 ^{er} janvier de l'an 1000 au 31 décembre 9999 après J.-C.	Sur 3 octets. L'affichage est au format 'YYYY-MM-DD'.
DATETIME	Dates et heures (de 0 h de la première date à 23 h 59 minutes 59 secondes de la dernière date).	Sur 8 octets. L'affichage est au format 'YYYY-MM-DD HH:MM:SS'.
YEAR[(2 4)]	Sur 4 positions : de 1901 à 2155 (incluant 0000). Sur 2 positions : de 70 à 69 (désignant 1970 à 2069).	Sur 1 octet ; l'année est considérée sur 2 ou 4 positions (4 par défaut). Le format d'affichage est 'YYYY'.
TIME	Heures de -838 h 59 minutes 59 secondes à 838 h 59 minutes 59 secondes.	L'heure au format 'HHH:MM:SS' sur 3 octets.
TIMESTAMP	Instants du 1 ^{er} Janvier 1970 0 h 0 minute 0 seconde à l'année 2037.	Estampille sur 4 octets (au format 'YYYY-MM-DD HH:MM:SS') ; mise à jour à chaque modification sur la table.

Données binaires

Les types BLOB (*Binary Large Object*) permettent de stocker des données non structurées comme le multimédia (images, sons, vidéo, etc.). Les quatre types de colonnes BLOB sont TINYBLOB, BLOB, MEDIUMBLOB et LONGBLOB. Ces types sont traités comme des flots d'octets sans jeu de caractère associé.

Type	Description	Commentaire pour une colonne
TINYBLOB(<i>n</i>)	Flot de <i>n</i> octets.	Taille fixe (maximum de 255 octets).
BLOB(<i>n</i>)		Taille fixe (maximum de 65 535 octets).
MEDIUMBLOB(<i>n</i>)		Taille fixe (maximum de 16 mégaoctets).
LONGBLOB(<i>n</i>)		Taille fixe (maximum de 4,29 gigaoctets).

3. Contraintes

Les contraintes ont pour but de programmer des règles de gestion au niveau des colonnes des tables. Elles peuvent alléger un développement côté client (si on déclare qu'une note doit être

comprise entre 0 et 20, les programmes de saisie n'ont plus à tester les valeurs en entrée mais seulement le code retour après connexion à la base ; on déporte les contraintes côté serveur).

Les contraintes peuvent être déclarées de deux manières :

- En même temps que la colonne (valable pour les contraintes monocolumnes) ; ces contraintes sont dites « en ligne » (*inline constraints*).
- Après que la colonne est déclarée ; ces contraintes ne sont pas limitées à une colonne et peuvent être personnalisées par un nom (*out-of-line constraints*).

Il est recommandé de déclarer les contraintes NOT NULL en ligne, les autres peuvent soit être déclarées en ligne, soit être nommées. Étudions à présent les types de contraintes nommées (*out-of-line*). Les quatre types de contraintes les plus utilisées sont les suivants :

CONSTRAINT *nomContrainte*

UNIQUE (*colonne1* [,*colonne2*]...)

PRIMARY KEY (*colonne1* [,*colonne2*]...)

FOREIGN KEY (*colonne1* [,*colonne2*]...)

REFERENCES *nomTablePere* [(*colonne1* [,*colonne2*]...)]

[ON DELETE {RESTRICT | CASCADE | SET NULL | NO ACTION}]

[ON UPDATE {RESTRICT | CASCADE | SET NULL | NO ACTION}]

CHECK (*condition*)

- La contrainte **UNIQUE** impose une valeur distincte au niveau de la table (les valeurs nulles font exception à moins que **NOT NULL** soit aussi appliquée sur les colonnes).
- La contrainte **PRIMARY KEY** déclare la clé primaire de la table. Un index est généré automatiquement sur la ou les colonnes concernées. Les colonnes clés primaires ne peuvent être ni nulles ni identiques (en totalité si elles sont composées de plusieurs colonnes).
- La contrainte **FOREIGN KEY** déclare une clé étrangère entre une table enfant (*child*) et une table père (parent). Ces contraintes définissent l'intégrité référentielle.
- La contrainte **CHECK** impose un domaine de valeurs ou une condition simple ou complexe entre colonnes (exemple : CHECK (note BETWEEN 0 AND 20), CHECK (grade='Copilote' OR grade='Commandant')).

Exemple :

4. Structure d'une table (DESCRIBE)

DESCRIBE (écriture autorisée DESC) est une commande qui vient de SQL*Plus d'Oracle et qui a été reprise par MySQL. Elle permet d'extraire la structure brute d'une table ou d'une vue.

Syntaxe

DESCRIBE *nomTable ou Vue*;

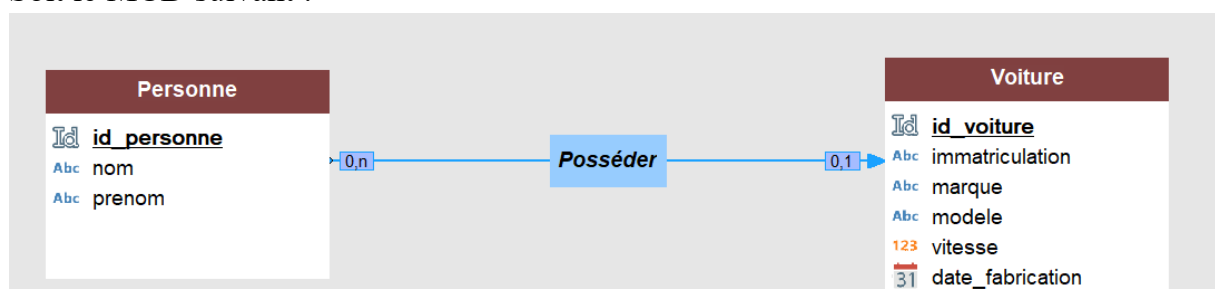
5. Commentaires

Dans toute instruction SQL (déclaration, manipulation, interrogation et contrôle des données), il est possible d'inclure des retours chariot, des tabulations, espaces et commentaires (sur une ligne précédée de deux tirets « - - », en fin de ligne à l'aide du dièse « # », au sein d'une ligne ou sur plusieurs lignes entre « /* » et « */ »). Les scripts suivants décrivent la déclaration d'une même table en utilisant différentes conventions :

Sans commentaire	Avec commentaires
<pre>CREATE TABLE Test(colonne DECIMAL(38,8)); CREATE TABLE Test (colonne DECIMAL(38,8));</pre>	<pre>CREATE TABLE -- nom de la table Test(#début de la description COLONNE DECIMAL(38,8)) -- fin, ne pas oublier le point- virgule. ; CREATE TABLE Test (/* une plus grande description des colonnes */ COLONNE /* type : */ DECIMAL(38,8));</pre>

6. Exercice d'application

Soit le MCD suivant :



- 1- Ecrire le MLD
- 2- Ecrire le script de création des tables en SQL.
- 3- Utiliser la commande « Describe » sur chaque table créée.

Corrigé :

```
CREATE TABLE Personne (  
    id_personne INT AUTO_INCREMENT PRIMARY KEY,  
    nom VARCHAR(50) NOT NULL,  
    prenom VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Voiture (  
    id_voiture INT AUTO_INCREMENT PRIMARY KEY,  
    immatriculation VARCHAR(20) NOT NULL UNIQUE,  
    marque VARCHAR(50),  
    modele VARCHAR(50),  
    vitesse INT NOT NULL,  
    date_fabrication DATETIME NOT NULL,    -- Date de fabrication du véhicule  
    id_personne INT NOT NULL,  
  
    -- Contrainte pour vérifier que la vitesse est réaliste (< 400 km/h)  
    CHECK (vitesse > 0 AND vitesse < 400),  
  
    -- Contrainte de clé étrangère : une voiture appartient à une seule personne  
    CONSTRAINT fk_voiture_personne  
        FOREIGN KEY (id_personne)  
        REFERENCES Personne(id_personne)  
        ON UPDATE CASCADE    -- Si l'ID d'une personne change, mise à jour automatique  
        ON DELETE RESTRICT   -- Impossible de supprimer une personne possédant une voiture  
);
```

Leçon 2 : Modification de la structure des tables

En SQL, la commande **ALTER** sert à modifier la structure d'une table existante (ou parfois d'une base). Elle ne sert pas à modifier les données, mais l'architecture de la table.

1. Ajouter une colonne

Syntaxe :

ALTER TABLE *NomTable*

```
ADD (    colonne1 type1    [NOT NULL | NULL] [DEFAULT valeur1] ,  
        Colonne2 type2    [NOT NULL | NULL] [DEFAULT valeur1] ,  
        );
```

Exemple:

```
ALTER TABLE client ADD (Revenu DECIMAL(7,2));
```

```
ALTER TABLE client ADD (  
    email        VARCHAR(4)    ,  
    ville        VARCHAR(30)    DEFAULT 'Abidjan' NOT NULL);
```

2. Renommer une colonne

Il faut utiliser l'option **CHANGE** de l'instruction **ALTER TABLE** pour renommer une colonne existante. Le nouveau nom de colonne ne doit pas être déjà utilisé dans la table. Le type (avec une éventuelle contrainte) doit être précisé.

Syntaxe :

```
ALTER TABLE NomTable CHANGE ancienNom    nouveauNom Type [NOT NULL |  
NULL] [DEFAULT valeur] [AUTO_INCREMENT] [UNIQUE [KEY] | [PRIMARY] KEY]  
[REFERENCES ...] [FIRST|AFTER nomColonne];
```

Exemple:

```
ALTER TABLE Pilote CHANGE ville adresse VARCHAR(30) AFTER Revenu;
```

3. Modifier une colonne (type, taille, options)

L'option **MODIFY** de l'instruction ALTER TABLE modifie le type d'une colonne existante sans pour autant la renommer. La syntaxe générale de cette instruction est la suivante, les options sont les mêmes que pour CHANGE :

Syntaxe :

```
ALTER TABLE NomTable MODIFY [COLUMN] nomColonneAmodifier typeMySQL  
[NOT NULL | NULL] [DEFAULT valeur]  
[AUTO_INCREMENT] [UNIQUE [KEY] | [PRIMARY] KEY]  
[REFERENCES ...] [FIRST|AFTER nomColonne];
```

Exemple:

```
ALTER TABLE client MODIFY adresse CHAR(20) NOT NULL;
```

4. Supprimer une colonne

L'option **DROP** de l'instruction ALTER TABLE permet de supprimer une colonne (aussi un index ou une clé que nous étudierons plus loin). La possibilité de supprimer une colonne évite aux administrateurs d'exporter les données, de recréer une nouvelle table, d'importer les données et de recréer les éventuels index et contraintes.

Syntaxe :

```
ALTER TABLE NomTable DROP  
[COLUMN] nomColonne ;
```

Exemple:

```
ALTER TABLE client DROP COLUMN adresse;
```

5. Ajouter des contraintes

La directive **ADD CONSTRAINT** de l'instruction **ALTER TABLE** permet d'ajouter une contrainte à une table. Il est aussi possible d'ajouter un index. La syntaxe générale est la suivante :

Syntaxe :

```
ALTER TABLE [nomBase].nomTable ADD  
  { INDEX [nomIndex] [typeIndex] (nomColonne1,...)  
  | CONSTRAINT nomContrainte typeContrainte }
```

Trois types de contraintes sont possibles :

- UNIQUE (*colonne1* [,*colonne2*]...)
- PRIMARY KEY (*colonne1* [,*colonne2*]...)
- FOREIGN KEY (*colonne1* [,*colonne2*]...)

```
REFERENCES nomTablePère (col1 [,col2]...)  
[ON DELETE {RESTRICT | CASCADE | SET NULL | NO ACTION}]
```

Exemple:

```
ALTER TABLE Avion ADD CONSTRAINT fk_Avion_comp_Compag  
FOREIGN KEY(proprio) REFERENCES Compagnie(comp);
```

6. Exercice d'application

Leçon 3 : Les suppressions dans le LDD

La commande **DROP** permet de **supprimer complètement** un objet de la base de données : Table, Vue, Index, Base de données.

1. Supprimer une table

Syntaxe :

```
DROP TABLE nom_table;
```

Exemple:

```
DROP TABLE Clients;
```

2. Supprimer les données d'une table

TRUNCATE efface toutes les lignes d'une table, mais garde la structure de la table.

Syntaxe :

```
TRUNCATE TABLE nom_table;
```

Exemple:

```
TRUNCATE TABLE Commandes;
```

3. Exercice d'application